

TERSE Tool Catalog (TTC)

Extension Specification v1.0

Author: Rudson Kiyoshi Souza Carvalho

Date: April 28, 2026

Status: Final — v1.0

Parent Spec: TERSE Format — DOI: 10.5281/zenodo.19058364

License: CC BY 4.0

Measured 66.6% reduction in LLM context tokens across 10 real MCP tool definitions — achieved by replacing verbose JSON Schema scaffolding with a compact, semantically annotated contract. Not pure compression: context-budget reallocation toward higher-value reasoning signals.

1. Motivation

Modern agentic systems communicate available tools to LLMs via structured JSON schemas defined by the Model Context Protocol (MCP). A typical MCP tool definition consumes between 100 and 270 tokens. A system with 50 installed tools may therefore place 5,000–13,500 tokens of tool catalog overhead into the context window — before any user instruction, memory, or reasoning occurs.

This overhead produces three concrete consequences:

- **Financial cost** — major LLM APIs offer prompt caching that reduces the per-request financial cost of static tool catalogs to approximately 10% of the base rate. Caching mitigates billing cost but does not eliminate the other two consequences below.
- **Attention cost** — cached tokens still occupy the context window and are processed by the LLM attention mechanism on every request. A large tool catalog competes with working memory, document context, and conversation history regardless of whether it is cached. This is especially critical for smaller and local models (Qwen 7B, Llama 3, Mistral) where context windows are narrow and prompt caching is unavailable.
- **Selection quality degradation** — LLMs exhibit lower tool-selection accuracy as catalog size grows, due to attention dilution across large tool inventories. This effect is independent of caching.

Beyond token cost, the MCP JSON Schema format has a structural limitation: it was designed as a machine-readable contract for tool execution, not as a semantic contract for tool selection. Critical information for LLM reasoning is either absent or buried in unstructured natural language description fields:

- No explicit trigger condition — the LLM must infer when to use a tool from a free-form description string.
- No error contract — failure modes are undeclared, making graceful degradation reasoning impossible.
- No retrieval taxonomy — there is no standard way to group or filter tools by domain, making dynamic tool retrieval harder to implement.

The TERSE Tool Catalog (TTC) extension addresses both problems simultaneously. It is not a compression of MCP JSON into a shorter format — it is a semantic reallocation of the context budget. TTC removes syntactic and documentary overhead that serves human readability but carries low reasoning signal for LLMs, then reinvests part of that saved budget into explicit tool-selection semantics that MCP JSON Schema does not provide.

The benchmark in Section 7 compares raw MCP JSON Schema definitions against semantically annotated TTC definitions. TTC adds **ERR**, **WHEN**, and **TAGS** — but omits verbose parameter descriptions and low-level JSON Schema constraints from the LLM context. The result is not lossless compression; it is semantic reallocation of the context budget.

Key claim: TTC achieves a measured 66.6% reduction in LLM context tokens by replacing verbose JSON Schema scaffolding and low-value parameter prose with a compact, semantically annotated contract. Although TTC introduces new fields — ERR, WHEN, and TAGS — the total representation remains substantially smaller because syntactic and documentary overhead are removed. The result is not pure compression, but context-budget reallocation toward higher-value reasoning signals.

2. Design Principles

TTC inherits all TERSE core principles:

- Density over verbosity — maximum information per token
- Determinism — identical input always produces identical output
- Human-readability — parseable by humans and machines alike
- Composability — tool blocks compose into server catalogs; catalogs compose into agent contexts

TTC adds principles specific to tool catalog representation as a semantic contract:

- Semantic trigger (**WHEN**) — each tool explicitly declares the condition under which the LLM should invoke it. This field does not exist in MCP JSON Schema, where trigger logic is buried in free-form description text. Making it a first-class field improves tool selection accuracy and reduces ambiguity.

- Error contract (ERR) — each tool explicitly declares its failure modes. This field does not exist in MCP JSON Schema. Without it, LLMs cannot reason about fallback strategies or graceful degradation when a tool fails.
- Retrieval taxonomy (TAGS) — each tool declares domain tags that support semantic grouping and dynamic retrieval. This field does not exist in MCP JSON Schema.
- MCP round-trip — the format supports lossless conversion to and from MCP JSON Schema for backwards compatibility.

The comparison is therefore not MCP JSON Schema vs. a compressed version of itself. It is MCP JSON Schema (verbose, semantically incomplete) vs. TTC (compact, semantically enriched). The 66.6% token reduction is achieved by removing syntactic overhead and reinvesting part of those savings in three fields — ERR, WHEN, TAGS — that MCP does not carry. The result is a net reasoning-signal gain, not a net information gain in the information-theoretic sense.

Field	MCP JSON	TTC	Role in LLM reasoning
Tool name & parameters	✓	✓	Basic execution contract
Parameter types	partial	✓	Input validation guidance
Intent (PURPOSE)	buried in description	✓ explicit	Tool selection clarity
Trigger condition (WHEN)	✗ absent	✓ explicit	Primary selection discriminator
Failure modes (ERR)	✗ absent	✓ explicit	Graceful degradation reasoning
Retrieval taxonomy (TAGS)	✗ absent	✓ explicit	Dynamic tool retrieval support
Verbose param descriptions	✓ verbose	✗ omitted	Human docs — no LLM signal

3. TOOL Block Syntax

3.1 Single Tool

```
TOOL <tool-id>
  PURPOSE: <single-line intent description>
  IN: <param-list>
  OUT: <return-list>
  ERR: <error-list>
  WHEN: <semantic trigger>
  TAGS: <tag-list>
```

Field reference:

Field	Required	Description
PURPOSE	Yes	One-line description of what the tool does
IN	Yes	Comma-separated typed input parameters
OUT	Yes	Comma-separated typed return values
ERR	Yes	Pipe-separated error codes
WHEN	Yes	Natural language trigger condition for LLM tool selection
TAGS	No	Comma-separated tags for grouping and retrieval

PURPOSE vs WHEN — implementation guideline

PURPOSE and WHEN are semantically adjacent but serve distinct roles. PURPOSE describes what the tool does (capability). WHEN describes the condition under which the LLM should invoke it (trigger). They should never be identical:

```
; Correct usage — PURPOSE states capability, WHEN states trigger condition
TOOL gmail_send_email
  PURPOSE: send_email via Gmail
  WHEN: user explicitly asks to send, compose, or deliver an email message

; Incorrect — WHEN is a copy of PURPOSE
TOOL gmail_send_email
  PURPOSE: send_email via Gmail
  WHEN: send_email via Gmail ; ← wrong: no trigger semantics
```

A practical rule: PURPOSE answers "what does this tool do?". WHEN answers "what must the user intend for me to call this tool?". WHEN should be specific enough to discriminate between similar tools in the same catalog.

3.2 Parameter Syntax

```
<param-id>:<type>[?]

; ? suffix marks the parameter as optional
; Multiple parameters are comma-separated
```

Supported types:

Type	Description
string	UTF-8 text
int	Integer number
float	Floating point number
bool	Boolean (true/false)
array[T]	Ordered list with optional element type, e.g. array[string]
object	Structured key-value object
any	Untyped — use when schema is dynamic

For tools that return structured data, use `object` or `array[object]` in the `OUT` field. These types cover the majority of real MCP tool responses:

```
; Simple scalar return
TOOL gmail_send_email
  OUT: message_id:string

; Structured object return
TOOL drive_read_file
  OUT: content:string, mime_type:string

; Collection return
TOOL gmail_read_inbox
  OUT: messages:array[object]

; When return schema is dynamic or version-dependent
TOOL legacy_api_call
  OUT: result:any
```

Detailed field-level schemas for object returns are intentionally deferred to the `EXAMPLE` block extension, planned for v1.2 (Section 12 — Future Extensions). The `OUT` field communicates shape, not schema — sufficient for LLM reasoning about downstream tool chaining.

3.3 Example: JSON Schema vs TTC

The following shows the same tool represented in MCP JSON Schema (current standard) and in TTC:

MCP JSON Schema (208 tokens measured)

```
{
  "name": "gmail_send_email",
  "description": "Sends an email message via the Gmail API to one or more recipients. Use this tool when the user explicitly requests to send, compose and send, or deliver an email message.",
  "input_schema": {
    "type": "object",
    "required": ["to", "subject", "body"],
    "properties": {
      "to": { "type": "string", "description": "Recipient address..." },
      "subject": { "type": "string", "description": "Subject line..." },
      "body": { "type": "string", "description": "Body content..." },
      "cc": { "type": "string", "description": "Optional CC..." }
    }
  }
}
```

TTC (55 tokens measured)

```
TOOL gmail_send_email
PURPOSE: send_email via Gmail
IN: to:string, subject:string, body:string, cc:string?
OUT: message_id:string
ERR: auth_failed | quota_exceeded | invalid_recipient
WHEN: user wants to send or compose an email
TAGS: gmail, email, communication
```

Same semantic content. 73.6% fewer tokens.

3.4 Normative WHEN Vocabulary

The WHEN field is authored in natural language. Without a controlled vocabulary, independent server authors may write semantically equivalent triggers using incompatible phrasing, degrading tool-selection accuracy in large catalogs. TTC v1.0 defines a normative vocabulary for WHEN authoring.

3.4.1 Required pattern

```
WHEN: user [intent-verb] to [action-verb] [object]

Intent verbs (choose one):
  wants | requests | asks | needs | intends

Action verbs (non-exhaustive controlled set):
  send | read | list | create | search | delete | update
  download | share | check | open | post | browse
  schedule | book | log | find | view | draft | upload
```

Both the intent verb and the action verb are required. The object (what is being acted on) should be specific enough to discriminate between similar tools in the same catalog.

3.4.2 Conformant examples

```
; Conformant
WHEN: user wants to send an email message
WHEN: user requests to list files in Google Drive
WHEN: user needs to create a calendar event
WHEN: user asks to search for a GitHub issue

; Non-conformant (missing intent verb or action verb)
WHEN: send email ; ← missing intent verb
WHEN: user email ; ← missing action verb
WHEN: user wants email ; ← missing action verb
```

3.4.3 Accuracy simulation

To validate the controlled vocabulary approach, a simulation was conducted using TF-IDF cosine similarity as an LLM tool-selection proxy. This is a deterministic approximation — not LLM inference — but it isolates the contribution of field structure to retrieval accuracy.

Setup: 12 tools, 36 user queries with ground-truth tool mappings, TF-IDF with bigrams. Two conditions: (A) MCP free-form description as selection signal, (B) TTC WHEN with controlled vocabulary as selection signal.

Condition	Accuracy	Correct / Total	Avg Confidence
(A) MCP free-form description	63.9%	23 / 36	0.098
(B) TTC WHEN — controlled vocabulary	72.2%	26 / 36	0.139
Delta	+8.3 pp	+3 correct	+0.041

The confidence delta (+0.041) is particularly relevant: higher confidence means the selection signal is stronger and less ambiguous, which reduces tool confusion in catalogs where multiple tools serve adjacent use cases.

Simulation caveat: TF-IDF cosine similarity approximates semantic retrieval but does not replicate LLM attention mechanisms. Real LLM benchmarks may show larger or smaller deltas. This simulation establishes directional evidence — controlled vocabulary improves retrieval signal — not a production performance claim.

4. MCP Server Catalog Syntax

Multiple tools from the same MCP server are grouped under an MCP block:

```
MCP <server-id> v<version>
  TOOL <tool-id>
    PURPOSE: ...
    IN: ...
    OUT: ...
    ERR: ...
    WHEN: ...

  TOOL <tool-id-2>
    ...
```


4.1 Agent Context Block

For agent initialization, tools from multiple servers compose into a single TOOLS context block. The optional [active/total] annotation communicates to the LLM that tool retrieval is active:

```
TOOLS v1.0 [3/47]
  MCP gmail v1.2
    TOOL gmail_send_email
    ...
  MCP google_drive v2.0
    TOOL drive_read_file
    ...
```

This pattern supports dynamic tool retrieval: the full catalog is stored compactly and only semantically relevant tools are injected per request.

5. ABNF Grammar Extension

This section extends the TERSE core ABNF grammar (DOI: 10.5281/zenodo.19058364):

```
tool-catalog = 1*mcp-block

mcp-block    = "MCP" SP server-id SP "v" version CRLF
              1*tool-block

server-id    = 1*ALPHA *( ALPHA / DIGIT / "_" / "-" )
version      = 1*DIGIT "." 1*DIGIT *( "." 1*DIGIT )

tool-block   = indent "TOOL" SP tool-id CRLF
              tool-purpose tool-in tool-out
              tool-err tool-when [ tool-tags ]

tool-id      = 1*ALPHA *( ALPHA / DIGIT / "_" )
indent       = 2*SP

tool-purpose = indent "PURPOSE:" SP 1*VCHAR CRLF
tool-in      = indent "IN:" SP param-list CRLF
tool-out     = indent "OUT:" SP param-list CRLF
tool-err     = indent "ERR:" SP error-list CRLF
tool-when    = indent "WHEN:" SP 1*VCHAR CRLF
tool-tags    = indent "TAGS:" SP tag-list CRLF

param-list   = param *( "," SP param )
param        = param-id ":" type [ "?" ]
type         = "string" / "int" / "float" / "bool" / "any"
              / "array" [ "[" type "]" ] / "object"

error-list   = error-code *( SP "|" SP error-code )
error-code   = 1*ALPHA *( ALPHA / DIGIT / "_" )

tag-list     = tag *( "," SP tag )
tag          = 1*ALPHA *( ALPHA / DIGIT / "_" / "-" )
```

6. Token Benchmark

Methodology: token counts measured using a BPE-compatible tokenizer (cl100k_base approximation) on 10 real MCP tool definitions representative of common agent configurations. Tokenizer used: regex-based BPE approximation consistent with GPT-4 / Claude tokenization.

6.1 Per-Tool Results

Tool	JSON Schema	TTC	Reduction
gmail_send_email	208	55	73.6%
gmail_read_inbox	121	52	57%
drive_list_files	141	53	62.4%
drive_read_file	117	53	54.7%
calendar_create_event	262	78	70.2%
slack_send_message	206	69	66.5%
github_create_issue	269	84	68.8%
notion_create_page	197	69	65%
jira_create_ticket	254	77	69.7%
web_search	173	60	65.3%
TOTAL (10 tools)	1948	650	66.6%

6.2 Projections

Extrapolated from per-tool averages measured above:

Catalog Size	JSON Schema (tokens)	TTC (tokens)	Reduction
5 tools	≈ 974	≈ 325	66.6%
20 tools	≈ 3,896	≈ 1,300	66.6%
50 tools	≈ 9,740	≈ 3,250	66.6%
100 tools	≈ 19,480	≈ 6,500	66.6%

At 100 tools: JSON Schema ≈ 19,480 tokens vs TTC ≈ 6,500 tokens per request. Absolute saving: ≈ 12,980 tokens per request. The absolute saving grows linearly with catalog size — the larger the catalog, the higher the ROI of adopting TTC.

7. Design Decisions and Trade-offs

7.1 Comparison framing

The correct comparison is not "MCP JSON Schema vs. TTC without the extra fields." The correct comparison is:

MCP JSON Schema (verbose, semantically incomplete) vs. TTC (compact, semantically enriched)

TTC achieves 66.6% token reduction while adding ERR, WHEN, and TAGS. A critic might argue: if you added fields, the savings come from removing constraints and descriptions, not from the format itself. This objection is partially correct, and worth addressing directly.

The claim is not that TTC achieves information gain. The claim is that TTC achieves reasoning-signal gain:

```
MCP JSON Schema original: ~195 tokens per tool
(nesting, quotes, property schemas,
 required arrays, verbose param descriptions)

TTC without new fields: ~35 tokens per tool
(PURPOSE + IN only)

TTC with all fields: ~65 tokens per tool
(PURPOSE + IN + OUT + ERR + WHEN + TAGS)

The 30-token reinvestment buys:
ERR → failure contract (absent from MCP)
WHEN → selection trigger (absent from MCP)
TAGS → retrieval taxonomy (absent from MCP)

Net: 195 → 65 tokens. Still -66.6%.
But the 65 tokens carry higher reasoning
signal than the original 195.
```

This is the core thesis: TTC does not reduce tokens by removing semantic content. It reduces syntactic and documentary overhead — verbose parameter descriptions, JSON Schema constraints, deep nesting — that serves human document readability but not LLM reasoning. It then reinvests part of the saved budget into explicit tool-selection semantics.

7.2 What TTC preserves from MCP

- Tool identity (tool-id) — stable reference for execution routing
- Parameter names, types, and optionality — sufficient for LLM to construct valid calls
- Return type shape (OUT) — enables downstream reasoning about tool output

7.3 What TTC adds over MCP

- **WHEN** — explicit semantic trigger condition; the primary field for tool selection reasoning. Without this, LLMs must parse trigger intent from free-form description text, introducing inconsistency and errors.
- **ERR** — explicit failure mode contract; enables LLM reasoning about fallback strategies and graceful degradation. Completely absent from MCP JSON Schema.
- **TAGS** — retrieval taxonomy; supports semantic grouping and dynamic tool retrieval (RAG over tools). Completely absent from MCP JSON Schema.

7.4 What TTC intentionally omits

- Verbose natural language parameter descriptions — replaced by typed names. The combination of typed parameter name + PURPOSE + WHEN provides sufficient signal for the LLM to construct valid calls without per-parameter prose.
- JSON Schema constraints (minLength, pattern, enum) — moved to server-side validation. The LLM needs to know what to pass, not the full validation ruleset. Servers validate on receipt. This separation of concerns reduces context overhead without sacrificing correctness.
- Example values — reserved for a future EXAMPLE block extension.

7.5 WHEN field: from limitation to normative contract

In pre-v1.0 drafts, the WHEN field was identified as a known limitation: natural language triggers authored independently by different server developers could overlap, conflict, or use incompatible vocabulary, degrading tool-selection accuracy in large multi-server catalogs.

TTC v1.0 addresses this through the normative WHEN vocabulary defined in Section 3.4. By requiring a controlled intent verb (wants | requests | asks | needs | intends) combined with a controlled action verb, inter-server consistency becomes structurally enforced rather than dependent on author discipline.

The accuracy simulation in Section 3.4.3 provides directional evidence: controlled vocabulary WHEN improves tool-selection accuracy by 8.3 percentage points and average selection confidence by 0.041 over free-form MCP descriptions in a 12-tool, 36-query benchmark. This validates the design decision to include WHEN as a normative field rather than deferring it to a future extension.

Remaining open item: the controlled action verb set in Section 3.4.1 is non-exhaustive. A normative annex listing the complete canonical verb set with definitions is planned for v1.1.

7.7 Relationship to prompt caching

Major LLM APIs (Anthropic, OpenAI) offer prompt caching for static context blocks, including tool catalogs. Cached tokens are billed at approximately 10% of the base rate when the catalog does not change between requests. This warrants a direct discussion of how caching interacts with the TTC value proposition.

Prompt caching mitigates the financial cost of large tool catalogs. It does not mitigate the attention cost. Cached tokens still occupy the context window and are processed by the attention mechanism on every request. The following consequences remain regardless of caching:

- Context window pressure — cached tool tokens compete with working memory, document context, and conversation history for the same fixed window. A 13,500-token catalog consumes 6.75% of a 200k window and 84% of a 16k window, cached or not.
- Selection quality degradation — attention dilution from large catalogs affects tool-selection accuracy independent of whether tokens are cached.
- Local and smaller models — Qwen 7B, Llama 3, Mistral, and similar models do not support prompt caching and operate with narrower context windows. TTC's compact representation is especially valuable in these deployments, which are common in on-premise and air-gapped agentic systems.
- Dynamic retrieval incompatibility — prompt caching requires a static tool block to produce cache hits. Systems that perform per-request tool retrieval (RAG over tools, as described in Section 8) produce different tool subsets per request, breaking cache locality. TTC's compact format enables dynamic retrieval at lower cost than MCP JSON Schema precisely because the per-tool token cost is lower.

Summary: prompt caching and TTC are complementary, not competing. Caching reduces the financial cost of static catalogs. TTC reduces the attention cost and enables dynamic retrieval patterns that caching cannot serve.

7.8 Validation responsibility

TTC follows a strict separation of concerns: schema constraints are a server concern, not a context concern. Servers validate on receipt; the LLM context carries only what is necessary for reasoning.

8. Relationship to Tool Retrieval

TTC is designed to work natively with dynamic tool retrieval (RAG over tools):

```
User message arrives
→ Semantic search over full catalog (stored as TTC in vector DB)
→ Top-K relevant tools selected (e.g. K=5 from 100 available)
→ Selected tools injected as TOOLS block into context
→ LLM sees only relevant subset
→ Cost: avg_ttc * K instead of avg_ttc * 100
```

TTC's compact format makes it practical to embed the full catalog in a vector database and achieve sub-linear context cost growth regardless of catalog size.

9. Reference Converter

Python reference implementation for MCP JSON Schema → TTC conversion. Available at github.com/RudsonCarvalho/terse-format.

```
def tool_to_ttc(tool: dict, indent: str = "  ") -> str:
    name = tool["name"]
    description = tool.get("description", "")
    input_schema = tool.get("input_schema", {})
    purpose = description[:80].rstrip()
    when = description.split(".")[0].strip()
    params = _convert_params(input_schema)
    return "\n".join([
        f"{indent}TOOL {name}",
        f"{indent}  PURPOSE: {purpose}",
        f"{indent}  IN: {params}",
        f"{indent}  OUT: result:any",
        f"{indent}  ERR: error",
        f"{indent}  WHEN: {when}",
    ])

def server_to_ttc(server_id, version, tools):
    header = f"MCP {server_id} v{version}"
    blocks = "\n\n".join(tool_to_ttc(t) for t in tools)
    return f"{header}\n{blocks}"
```

Round-trip fidelity note

The reference converter above is illustrative. It generates sensible defaults for OUT (result:any) and ERR (error) by extracting what is inferable from MCP JSON Schema. Full round-trip fidelity — where TTC can be converted back to an MCP JSON Schema without information loss — requires explicit server-side annotation of four fields that MCP JSON Schema does not carry natively:

- OUT — the return type contract must be declared by the server author, not inferred from description text.
- ERR — failure modes must be declared by the server author. MCP JSON Schema has no error contract field.
- WHEN — the semantic trigger must be authored explicitly. The converter approximates it from the first sentence of the description, which is often imprecise.
- TAGS — the retrieval taxonomy must be declared by the server author.

In practice, TTC is not a lossless compression of MCP JSON Schema — it is an enriched reformulation. The four fields above represent new information that must be provided by the server author during TTC annotation. The converter handles the mechanical transformation; the semantic enrichment requires human or LLM-assisted authoring.

11. Future Extensions

Version	Extension	Description
v1.1	WHEN verb annex	Normative canonical verb set with definitions for WHEN authoring
v1.1	ALIAS field	Alternative trigger phrases for improved retrieval recall across vocabulary variants
v1.2	EXAMPLE block	Input/output examples for few-shot tool use
v1.2	COST annotation	Estimated token/latency cost per tool call
v1.3	AUTH annotation	Required OAuth scope declaration
v1.3	CHAIN annotation	Declares tool dependencies and composition patterns
v2.0	DEPRECATED flag	Marks tools pending removal with migration path

12. References

- Carvalho, R. K. S. (2025). TERSE Format Specification. Zenodo. <https://doi.org/10.5281/zenodo.19058364>
- Anthropic. (2024). Model Context Protocol Specification. <https://modelcontextprotocol.io/spec>